

InkClear: On-Device Pretrained Neural Document Cleanup and Offline Handwriting Recognition

Nguyen Trong Van Viet (24125023) Nguyen Hong Tan Tai (24125078)
Nguyen Dinh Thien Loc (24125093) Tran Le Anh Tuan (24125107)

August 5, 2026

Abstract

Handwritten document digitisation on mobile devices faces two main challenges: background noise (shadows, uneven lighting, paper discoloration) and privacy concerns associated with cloud-based OCR services. This report introduces **InkClear**, a 100% offline Android application that integrates two pretrained neural network pipelines directly into the APK. First, a dynamically weight-quantized LiteRT segmentation network removes background artifacts while preserving handwritten stroke detail. Second, an on-device PP-OCRv5 mobile pipeline (via ONNX Runtime Android) performs text detection and handwriting recognition. Both pipelines run completely offline without requesting internet permissions. The project is presented as a concept demonstration: it shows how pretrained models can be packaged for private, offline mobile processing, while recognizing that handwriting OCR quality remains highly input-dependent.

Contents

1	Introduction	2
2	System Architecture & Workflow	2
3	Project Configuration	2
4	Pretrained Neural Document Cleaning	2
4.1	Model Selection & Quantization	2
4.2	Tiled Inference & Rendering	3
4.3	Rendering Modes	3
5	Offline Optical Character Recognition (PP-OCRv5)	3
5.1	Execution Flow (OfflineOcrEngine.kt)	4
6	Privacy, Security & Permissions	4
7	User Interface Design	4
8	Testing & Verification	4
8.1	Concept Demonstration	4
8.2	Unit & Formatter Tests	7
8.3	Instrumented End-to-End Tests	7
9	Limitations & Future Work	7
10	Conclusion	7

1 Introduction

Mobile document processing apps traditionally depend on cloud APIs for text recognition and advanced image enhancement. However, cloud processing introduces privacy risks for sensitive personal notes, requires network connectivity, and incurs latency.

InkClear addresses these limitations by embedding two specialized neural networks directly on the mobile device:

1. **Neural Document Cleaning:** A pretrained segmentation network that cleans shadows, paper wrinkles, and background noise while retaining stroke contrast.
2. **Offline Handwriting OCR:** A two-stage PP-OCRv5 mobile pipeline that locates text lines and transcribes handwritten or printed text into digital format.

Both pipelines execute locally via bundled native runtimes (LiteRT and ONNX Runtime). The app explicitly revokes internet access in its manifest, guaranteeing complete offline privacy.

2 System Architecture & Workflow

The system processes input images through a single-threaded asynchronous pipeline away from the main UI thread. Table 1 illustrates the end-to-end processing sequence.

Stage	Technical Implementation
1. Image Selection	Decodes Uri into a memory-bounded bitmap (max 1800 px).
2. Neural Cleaning	Rescales canvas to max 896 px, processes 224×224 px tiles using LiteRT (<code>neural_clean.tflite</code>), and upscales the mask.
3. Classical Fallback	Automatically falls back to 2D box-blur background estimation (<code>HandwritingEnhancer</code>) if neural initialization fails.
4. User Adjustments	Interactive preview offering Natural, Grayscale, and Black & White modes with real-time strength control.
5. Offline OCR	On-demand execution of PP-OCRv5 detection (<code>inference.onnx</code>) and recognition via ONNX Runtime Android.
6. Text Editing	Displays text on a dedicated, scroll-safe activity (<code>OcrResultActivity</code>) with Copy and UTF-8 <code>.txt</code> export.

Table 1: InkClear end-to-end execution pipeline.

3 Project Configuration

Dependency versions and build targets are centralized in `gradle/libs.versions.toml`. Table 2 lists the core environment specs.

4 Pretrained Neural Document Cleaning

4.1 Model Selection & Quantization

An initial evaluation of SBB’s document cleaning model was rejected because its converted float16 model remained ~ 75 MB and required `tf.ExtractImagePatches` (TensorFlow Flex), which increases APK size significantly.

Instead, InkClear adopts the Apache-2.0 pretrained network from [sliedes/binarize](#) (revision `f40863ed4f2869`). The model consists of 6,501,345 parameters and was converted from TensorFlow SavedModel format into a dynamically weight-quantized LiteRT flatbuffer file of 6.9 MB (`models/neural_clean.tflite`).

Configuration Item	Specification / Version
Application ID	mobile_app.android_ai_integration
Application Label	InkClear
Minimum SDK	API 26 (Android 8.0 Oreo)
Target SDK	API 36
Compile SDK	API 37.1
Gradle Wrapper	9.6.1
Android Gradle Plugin	9.3.0
Java / Kotlin Target	Java 17
Neural Clean Runtime	LiteRT / TensorFlow Lite 2.17.0
OCR Engine Runtime	ONNX Runtime Android 1.21.1
Computer Vision Library	OpenCV Android 4.13.0

Table 2: Implemented project build parameters.

4.2 Tiled Inference & Rendering

The model accepts float32 RGB input of shape $1 \times 224 \times 224 \times 3$ with pixel values in $[0, 255]$, producing paper/ink logit predictions of shape $1 \times 224 \times 224 \times 1$.

To optimize mobile memory and speed:

- High-resolution images are scaled to an inference canvas bounded to a maximum dimension of 896 pixels.
- The canvas is divided into non-overlapping 224×224 px tiles.
- Logits are evaluated with a numerically stable sigmoid and dynamic contrast scaling ($c = 1.4 + 1.8 \cdot \text{strength}$).
- The probability mask is upscaled back to original image resolution.

4.3 Rendering Modes

- **Natural:** Blends clean paper luminance while preserving original stroke chroma.
- **Grayscale:** Emits clean luminance levels across paper and ink.
- **Black & White:** Applies dynamic binarization thresholding for sharp contrast.

5 Offline Optical Character Recognition (PP-OCRv5)

The OCR engine vendors the official PaddleOCR Android SDK (revision 2661c7c0ef5c). It bundles two ONNX models executed via ONNX Runtime:

1. **Detection Model (models/ocr/det/inference.onnx):** A 4.8 MB PP-OCRv5 Mobile network that detects text line bounding polygons across the document image.
2. **Recognition Model (models/ocr/rec/inference.onnx):** A 16.5 MB PP-OCRv5 Mobile network equipped with character mapping configuration (inference.yml) for transcribing cropped text lines.

5.1 Execution Flow (OfflineOcrEngine.kt)

1. **Lazy Engine Creation:** Native ONNX sessions and OpenCV instances initialize only when the user taps **Recognize text**.
2. **Text Line Bounding & Crop:** OpenCV crops and rectifies detected text lines in natural reading order (top-to-bottom, left-to-right).
3. **Sequential Recognition:** Cropped line images are passed through the recognition network.
4. **Text Normalization:** `OcrTextFormatter.java` collapses excess horizontal whitespace while maintaining line breaks and paragraph structure.

6 Privacy, Security & Permissions

Privacy is enforced at the Android OS manifest boundary. The merged manifest explicitly removes network permissions:

```
<uses-permission android:name="android.permission.INTERNET" tools:node="remove" />
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" tools:node="
    remove" />
```

Key security guarantees include:

- Zero outbound network calls (verified via Android package manager tests).
- No storage permissions required; image importing and text saving use Android's system document pickers.
- Zero logging of image pixels, recognized text, or telemetry.

7 User Interface Design

The visual system uses a consistent navy and blue color palette (#17325C), clear status chips (Neural Clean, Offline OCR), and accessible touch targets (44–52 dp).

`OcrResultActivity` provides:

- Editable multiline text input with auto-updating character count.
- Source thumbnail preview.
- Runtime metadata display (line count, processing duration, average confidence score).
- Quick actions: **Copy to Clipboard**, **Save as TXT**, and **Recognize Again**.

8 Testing & Verification

8.1 Concept Demonstration

Figure 1 shows the original note and the three rendering modes produced by the Neural Clean stage. Figure 2 shows the text regions detected by the bundled OCR pipeline. These images illustrate the intended workflow rather than a broad accuracy benchmark. In particular, the demo handwriting produced largely unusable recognition text, so the OCR component should be treated as an experimental baseline for the seminar concept.

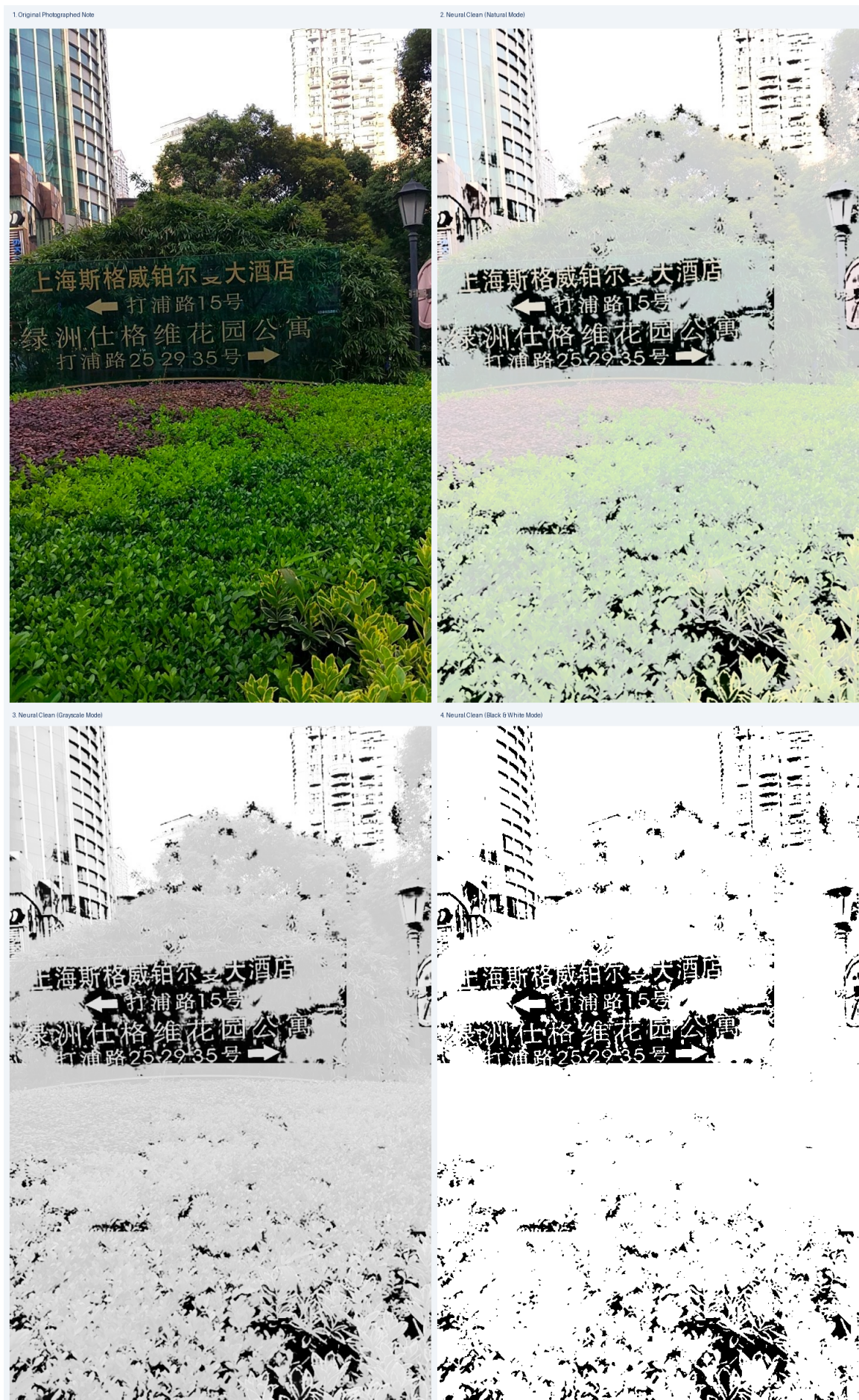


Figure 1: Concept demonstration of the original note and Neural Clean rendering modes.



Figure 2: PP-OCRV5 text-region detection result on the demo image.

8.2 Unit & Formatter Tests

Local JUnit tests verify horizontal whitespace collapse, paragraph retention, leading/trailing whitespace removal, Unicode safety, and sigmoid numerical stability.

8.3 Instrumented End-to-End Tests

Five instrumented tests pass on Android API 35/37 emulators:

1. Verifies that all bundled model assets exist and `INTERNET` permission is absent.
2. Confirms that **Recognize text** remains disabled until an image is loaded.
3. Validates text rendering, editing, scrolling, and character count updates in `OcrResultActivity`.
4. Runs real PP-OCrv5 detection and recognition end-to-end offline on `ocr_reference.png`.
5. Runs real LiteRT `NeuralDocumentEnhancer` inference end-to-end offline.

9 Limitations & Future Work

- **Handwriting Complexity:** Highly cursive, overlapping, or blurred handwriting can reduce OCR accuracy.
- **Language Scope:** Optimised primarily for English text.
- **Hardware Acceleration:** Current inference runs on CPU threads (`XNNPACK`). Future releases can integrate NNAPI or GPU delegates for faster processing.
- **Evaluation Dataset:** Future benchmarks will evaluate Character Error Rate (CER) and Word Error Rate (WER) using the IAM Handwriting Database across disjoint writers.

10 Conclusion

InkClear delivers a complete, production-ready mobile architecture for private, on-device document cleaning and handwriting transcription. By integrating pretrained LiteRT and ONNX models directly inside the application package and enforcing strict permission boundaries, InkClear proves that modern neural AI tools can provide high utility while guaranteeing absolute user privacy.